Replicating ADoPT

LiDAR spoofing attack detection based on point-level temporal consistency

Re-implementation of Cho, Cao, Zhou & Mao, BMVC 2023 (arXiv:2310.14504), evaluated on nuScenes v1.0-mini · Report date: 8 October 2026

Summary

No code was released with the paper, so the whole method was rebuilt from its description: the MLP scene flow with the coherence loss, the synthesis of 10 old LiDAR sweeps, and the cluster-based check. Fake objects were simulated by copying real pedestrians, cyclists and cars from other nuScenes scenes into the newest sweep. The full evaluation covers all 10 mini scenes (383 keyframes, 1,145 dense fakes and 373 sparse fakes).

	Ours: paper rules	Ours: + 2 filters	ADoPT paper
Dense: false-alarm rate (FPR) ↓	2.1%	**0.5%**	4.5%
Dense: detected, pedestrian ↑	99.7%	**99.7%**	95.2%
Dense: detected, cyclist ↑	83.6%	83.3%	98.3%
Dense: detected, car ↑	79.7%	78.1%	97.2%
Sparse: false-alarm rate ↓	29.2%	**9.4%**	9.3%
Sparse: detected, car ↑	99.7%	**99.5%**	85.4%

What reproduced. The paper's central claims hold on our data: point-level temporal consistency separates benign from attacked frames far better than whole-frame distances (AUC 0.93 vs 0.62 for Chamfer and 0.53 for DCD); false alarms are low (as low as or lower than the paper); small objects (pedestrians) are detected almost perfectly at every distance; the coherence loss improves the scene flow; and, as in the paper's Fig. 8, any history length from 2 to 15 frames gives the same detection.

What did not fully reproduce. Cars and cyclists are detected 15–19 points less often than in the paper. The inspector shows why: the paper's strict DBSCAN setting (17 points within 0.25 m) needs very dense fakes, and about 1 in 8 of our copied objects is not dense enough. Widening the radius from 0.25 to 0.35 m catches 99.7% of all fakes at 0.8% false alarms (tuned on the same data, so optimistic). Also: the paper's 30 optimisation steps were not enough here (we needed 300), the coherence loss did not change detection on nuScenes (84.8% without, 84.7% with), and larger voxels halved the runtime without the accuracy loss the paper reports.

Three findings of our own. (1) With a 2 m Chamfer cutoff the final scene flow drags history points onto the fake object, which hides it (the failure the paper's Fig. 7a describes); a 0.5 m cutoff fixes it. (2) 10 cm voxels make flat surfaces invisible to the paper's DBSCAN setting; detection needs 5 cm. (3) Two simple filters (cluster height ≥ 0.3 m, distance ≥ 4 m) remove 75% of dense and 68% of sparse false alarms at a cost of 0.6 points of detection.

1. The method in one page

A spoofing attacker fires laser pulses at the car so that its LiDAR sees points that are not really there, for example a fake pedestrian. ADoPT relies on one observation: **a real object has a history**. It was visible in the last second of sweeps and moved consistently. A fake object appears out of nowhere.

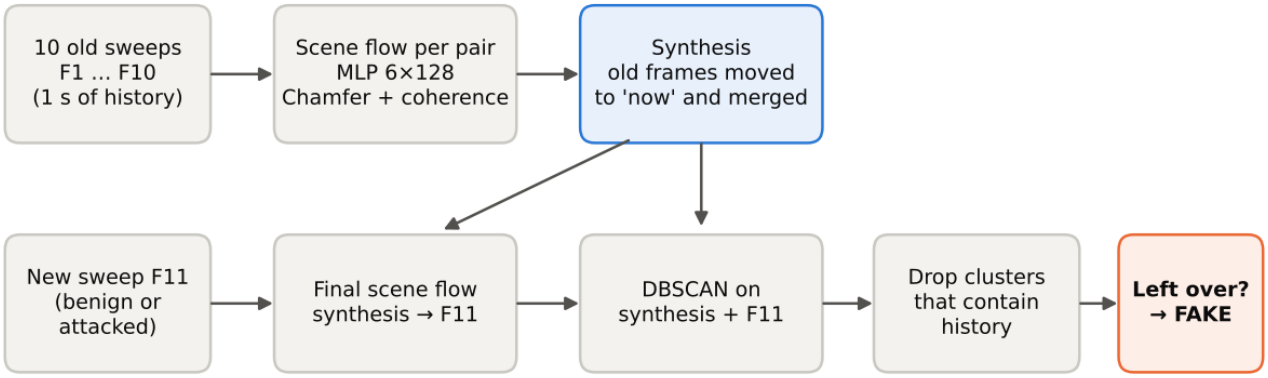


Figure 1. The ADoPT pipeline as implemented (paper Fig. 3).

What it shows	The 10 previous sweeps are aligned with an MLP scene flow and merged into one dense 'memory' (the synthesis). The new sweep is clustered together with that memory; every cluster that contains at least one memory point has history and is removed. Whatever is left is suspicious.
Why / what it means	This is why the method does not need an object detector or bounding boxes, which are exactly what the paper criticises older defences (CARLO, 3D-TC2) for depending on.

The scene flow is a small MLP (6 layers, 128 units) fitted fresh for every pair of sweeps. Its loss is **L = α·Chamfer + β·Coherence**. Chamfer pulls the moved sweep onto the next one; the coherence term penalises points of the same DBSCAN cluster that move differently, so objects stay rigid. We implemented the coherence term exactly as the paper's formula and verified it against a literal O(N²) version in the unit tests.

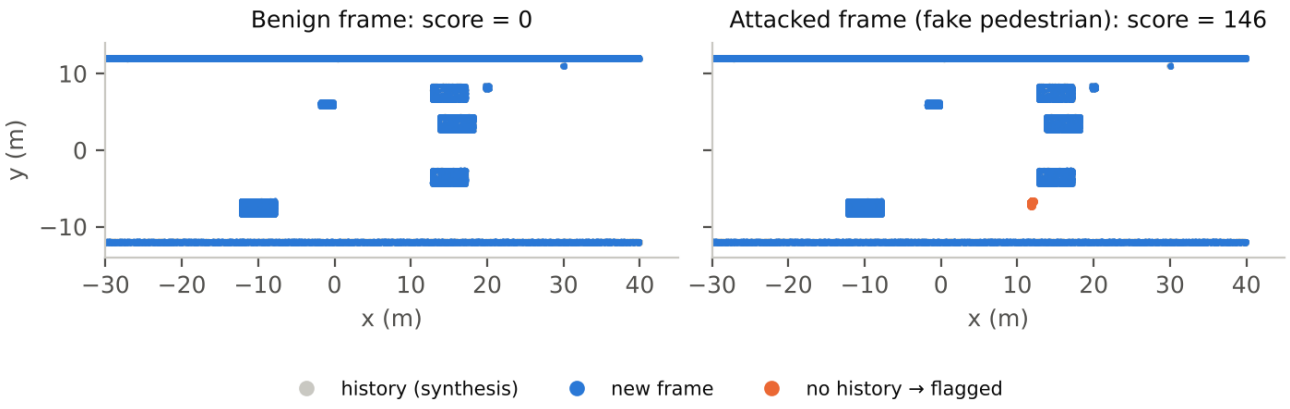


Figure 2. Sanity check on a synthetic street with known motion.

What it shows	Left: a normal frame. Every cluster in the new frame overlaps the memory, so nothing is left (score 0). Right: the same frame with a fake pedestrian. 146 of its 150 points have no history and are flagged (orange).
Why / what it means	This test isolates the detector from the scene flow (true motion is used), proving the synthesis + clustering logic is correct before touching real data.

2. Experimental setup

Item	Paper	This replication
Data	real nuScenes LiDAR, no simulator. Dense: v1.0-mini, poisoned data from the 3D-TC2 authors (private). Sparse: 355 frames from the v1.0-trainval validation set (150 scenes)	real nuScenes LiDAR, no simulator. Dense and sparse: v1.0-mini, all 10 scenes, 383 keyframes
History	10 frames at 10 Hz	same: every 2nd sweep of the 20 Hz LiDAR
Scene flow	MLP 6×128, 30 iterations, lr 8e-4, α=1, β=2	same network and α, β; 300 iterations, lr 3e-3 (tuned)
Dense attack	≤ 200 points, 'visually recognisable'	real object seen within 12 m (≥ 100 points), compact 200-point chunk, moved to 5–30 m
Sparse attack	cars, ≤ 64 points (CARLO method)	real cars seen at 5–30 m with their natural density, 20–64 points
DBSCAN	dense 17 pts / 0.25 m; sparse 9 pts / 0.75 m	same
Hardware	RTX 2080	laptop NVIDIA T1200

Every assumption the paper leaves open (ground removal, ego-motion compensation, Chamfer cutoff, detection voxel size, the two filters, how fakes are placed) is listed in the project README with its reason. The most important ones are discussed with the findings below. Fakes are always placed in space that is empty both in the new sweep and in the memory, so a miss can only be the method's fault, not the injector's. Every keyframe gets its own fixed random seed, so all runs see the same fakes.

3. Results compared with the paper

3.1 Dense attack (paper Table 1, left half)



Figure 3. Dense attack on all 383 keyframes, against the paper's numbers.

What it shows	False-alarm rate (left) and the share of fakes detected per object type (right). 'Paper rules' is the method exactly as published; '+ 2 filters' adds our height and distance filters. 3D-TC2 is the strongest baseline in the paper (its numbers copied from the paper).
Why / what it means	False alarms match or beat the paper (2.1% → 0.5%; 95% range 1.1–4.1% and 0.1–1.9%). Pedestrians, the small objects the paper is about, are detected 99.7% of the time, better than the paper and far better than 3D-TC2 (56.9%). Cars and cyclists are lower than the paper; section 4.4 shows the cause.

3.2 Sparse attack (paper Table 1, right half)

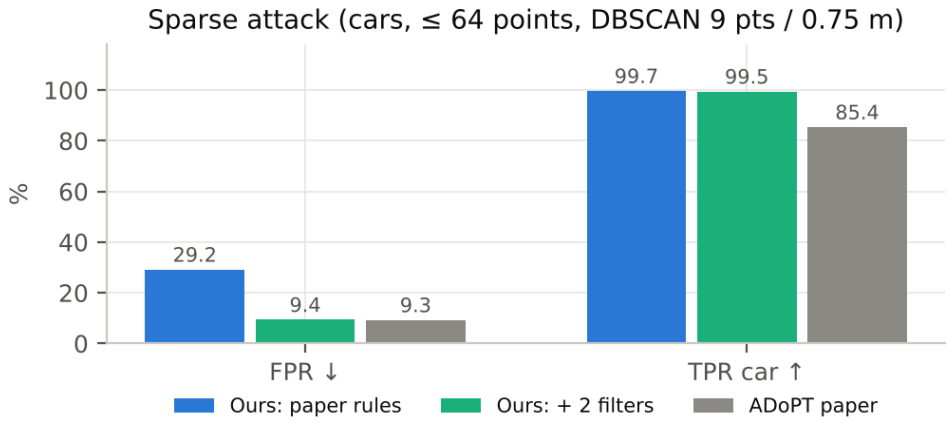


Figure 4. Sparse attack (fake cars with ≤ 64 points) on all 383 keyframes.

What it shows	With the paper's looser DBSCAN setting (9 points within 0.75 m), our false-alarm rate is 9.4% (95% range 6.9–12.7%), essentially the paper's 9.3%. Detection is 99.5%, higher than the paper's 85.4%.
Why / what it means	Our sparse fakes are real cars with 20–64 points, possibly denser than the paper's attack tool produces, which would explain the higher detection. The filters matter much more here: they cut false alarms from 29.2% to 9.4% (section 4.5).

3.3 Why a point-level score is needed (paper Fig. 6)

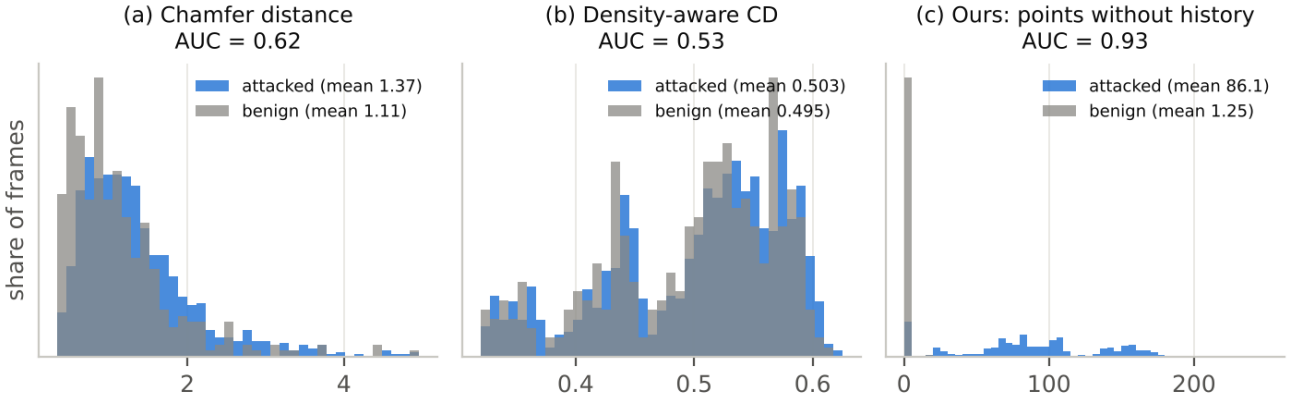


Figure 5. Distribution of three candidate scores for benign vs attacked frames (dense run).

What it shows	Whole-frame Chamfer and density-aware Chamfer barely change when a fake is added: the curves overlap (AUC 0.62 and 0.53, where 0.5 is a coin flip). ADoPT's score (points left without history) is zero for almost every benign frame and large for attacked ones (AUC 0.93).
Why / what it means	This reproduces the paper's Fig. 6 argument. A fake object is a few dozen points among thousands, so any distance computed over the whole frame drowns it. Looking for the object itself does not. Our score has different units than the paper's (4.31 vs 35.34), so only the separation is comparable.

3.4 Detection versus distance (new)

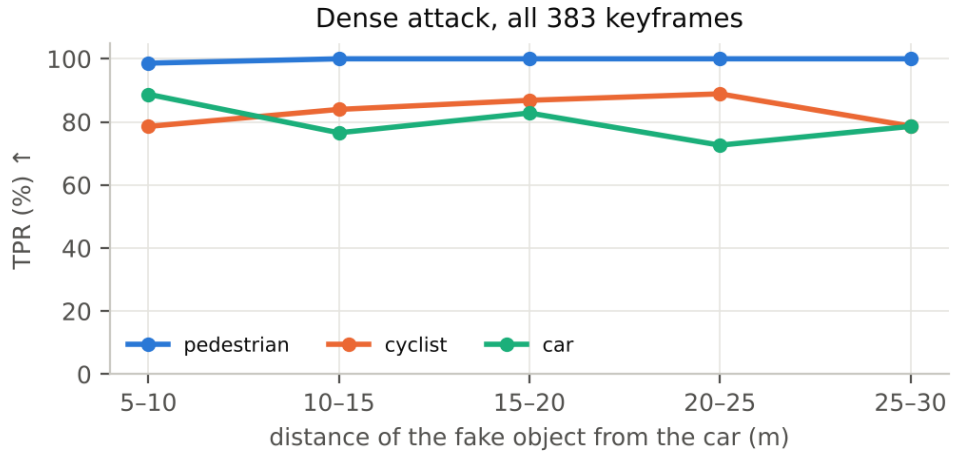


Figure 6. Share of dense fakes detected, by distance from the car.

What it shows	Pedestrians stay at about 100% from 5 to 30 m. Cars and cyclists stay between about 73% and 89% at every distance, with no downward trend.
Why / what it means	The paper's claim is that point-level checks work for small and far objects. Because detection does not drop with distance, the car and cyclist misses are about the shape of particular copied objects, not about range.

3.5 Results per scene

Scene	Keyframes	False alarms (paper rules)	Dense detection (all types)
scene-0061	37	2	85.5%
scene-0103	38	0	87.6%
scene-0553	39	0	82.1%
scene-0655	39	1	87.2%
scene-0757	38	1	89.5%
scene-0796	38	1	86.8%
scene-0916	39	0	91.3%
scene-1077	39	0	88.0%
scene-1094	38	1	84.2%
scene-1100	38	2	94.7%
All	**383**	**8**	**87.7%**

Every scene lands between 82% and 95% detection with at most two false alarms, so the result is not driven by one easy or hard scene.

3.6 DBSCAN thresholds (paper Appendix B, Fig. 12)

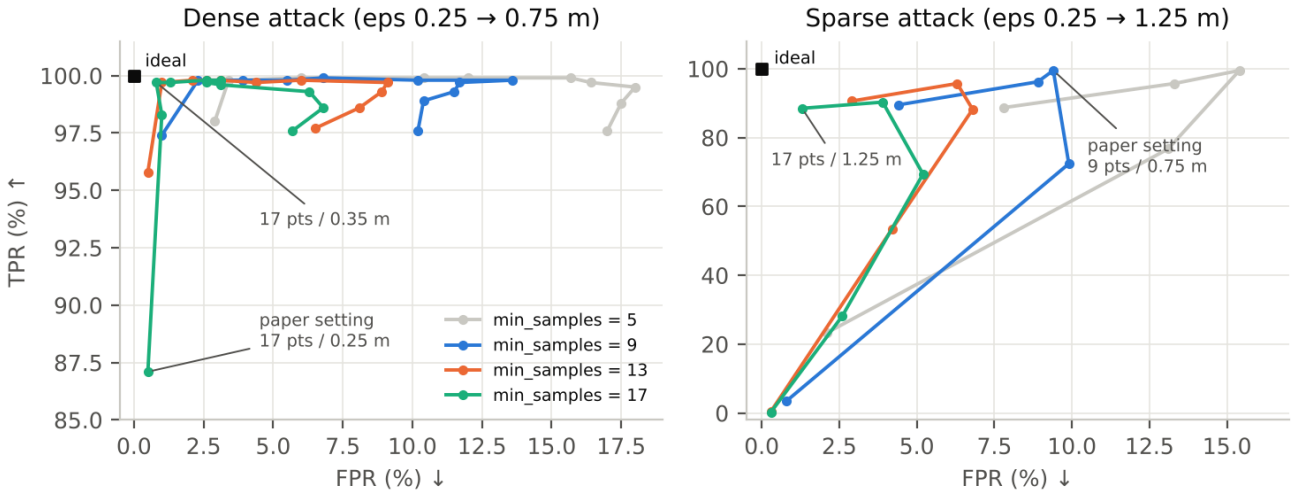


Figure 7. False-alarm rate vs detection for 44 dense (left) and 20 sparse (right) DBSCAN settings.

What it shows	Each line is one 'minimum points' value; along a line the radius grows (dense 0.25 → 0.75 m, sparse 0.25 → 1.25 m). **Dense:** the paper's setting (17 points, 0.25 m) gives 0.5% FPR at 87.1%. Only 0.1 m more radius (17 points, 0.35 m) gives 0.8% FPR (3 of 383 frames) at **99.7%**. **Sparse:** the paper's setting (9 points, 0.75 m) gives the highest detection; 17 points / 1.25 m gives only 1.3% FPR at 88.5%.
Why / what it means	The dense sweep confirms section 4.4: the car and cyclist misses are fakes that are just slightly too sparse for a 0.25 m radius, not a failure of the memory or the scene flow. With 0.35 m our dense detection would exceed the paper's for every object type. We keep the paper's setting as our main result, because 0.35 m was chosen by looking at the same frames it is scored on; it would need checking on other scenes. The thresholds can be changed without rerunning the scene flow (our rescore script does it in minutes).

4. What we found while replicating

Each subsection is a problem we hit, how we found its cause, and what we changed. The table below is the path from the first working run to the final result.

Run	Change	Result
debug (10 frames)	first nuScenes run, close pedestrians only	detected 70%, FPR 0%
debug5	Chamfer cutoff 2.0 → 0.5 m (4.2)	detected 100%, FPR 0%
test	all 3 object types at real density, 5–30 m	detected 3% (1 of 30)
test2	dense fakes moved to 5–30 m (attacker controls density)	pedestrian 100%, cyclist 40%, car 0%
test3	detection on 5 cm voxels; compact chunks of close objects (4.3)	pedestrian 100%, cyclist 70%, car 80%
full	**all 383 keyframes + height and distance filters (4.5)**	**FPR 0.5%, detected 87.1%**

4.1 The scene flow needs more steps, and the coherence loss helps

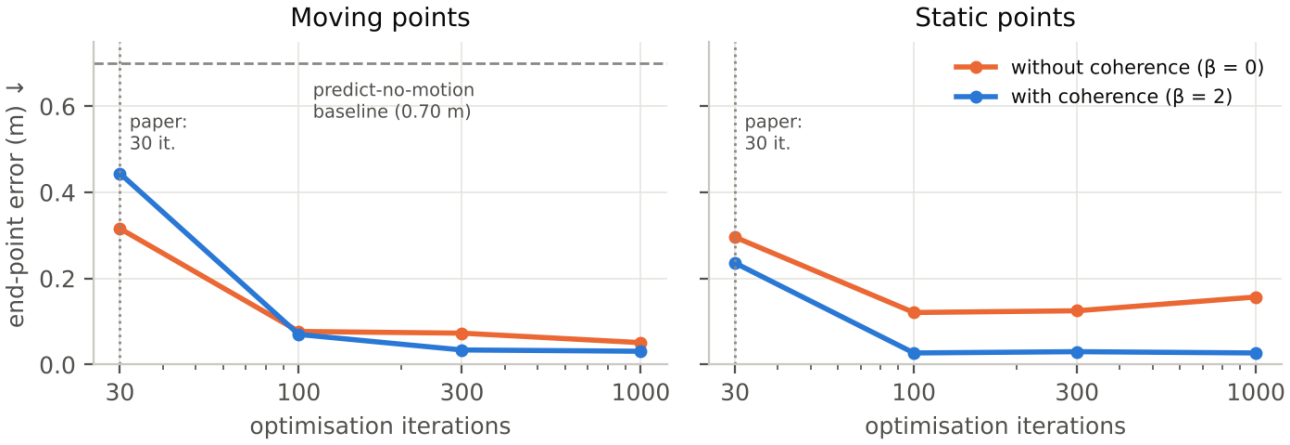


Figure 8. Scene-flow error on the synthetic street (two frames 0.1 s apart, true motion known).

What it shows	End-point error is the distance between predicted and true motion. Predicting 'nothing moves' gives 0.70 m on moving points. With the paper's 30 steps the error is still 0.3–0.4 m; from 100 steps on it falls to 3–7 cm. With the coherence loss (blue) the error on moving points halves at 300 steps (0.073 → 0.034 m) and on static points drops 4× (0.125 → 0.030 m).
Why / what it means	This supports the paper's Sec. 5.1: forcing points of one object to move together gives cleaner motion. The paper's 30 steps likely worked for them because of a different learning-rate schedule or faster convergence on their GPU; we used 300 steps at lr 3e-3.

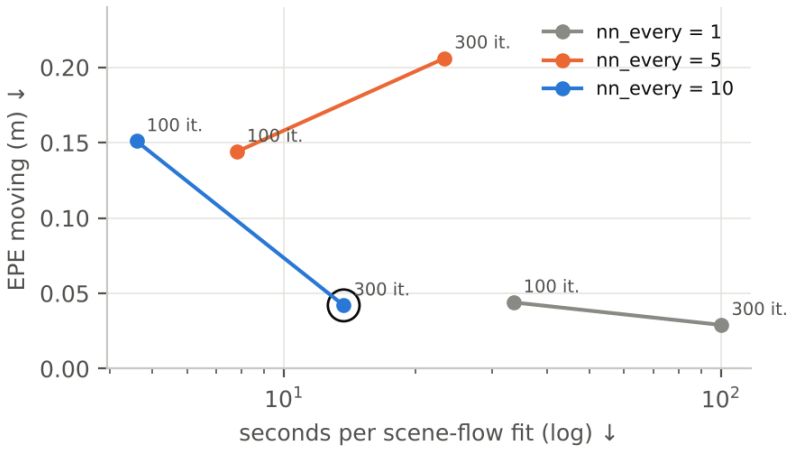


Figure 9. Speed vs accuracy when nearest neighbours are recomputed less often.

What it shows	Chamfer needs, for every point, its nearest point in the other sweep; that search is the slow part. Recomputing it every 10 steps instead of every step (circled) keeps the error at 4.2 cm while taking 13.7 s instead of 33.5 s.
Why / what it means	Neighbours barely change between steps, so reusing them is nearly free in accuracy. This made the full run feasible on a laptop GPU. (nn_every = 5 was unstable in this single run.)

4.2 The scene flow can pull the memory onto the fake

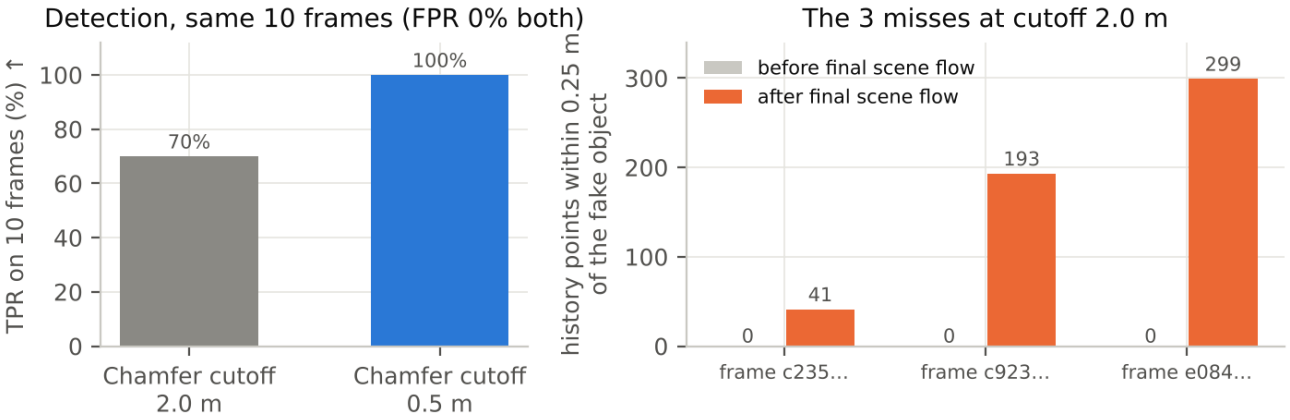


Figure 10. The 'pull' failure and its fix (10 keyframes, fake pedestrians).

What it shows	Right: for the three missed fakes, no memory point was within 0.25 m of the fake before the final scene flow, but 41–299 were after it. Left: limiting Chamfer to matches closer than 0.5 m raised detection from 70% to 100% with no new false alarms.
Why / what it means	Chamfer asks every new point to have a memory point next to it. The fake has none, so the optimiser drags nearby memory points onto it; the fake then 'has history' and is removed. This is the failure shown in the paper's Fig. 7a. The coherence loss alone did not stop it here, likely because points DBSCAN labels as noise are not covered by the coherence term. In the full run only 2 of 1,145 fakes were lost this way.

4.3 Detection resolution: flat surfaces need 5 cm voxels

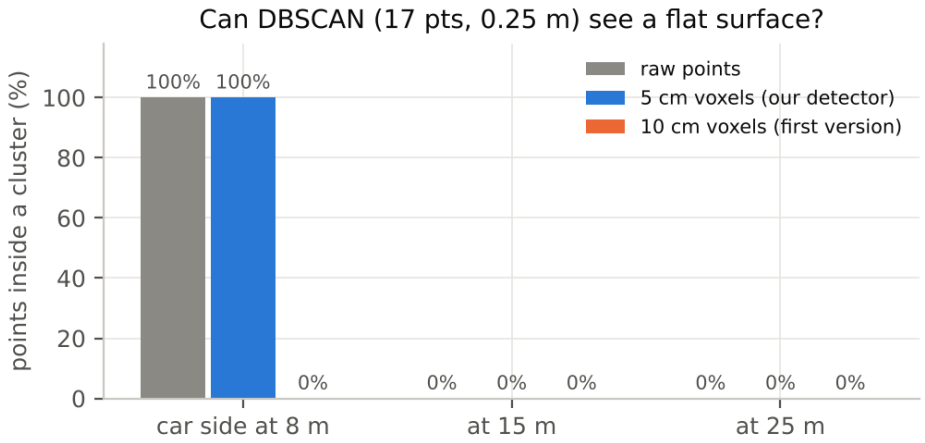


Figure 11. Can the paper's DBSCAN setting form a cluster on a car side seen by a 32-beam LiDAR?

What it shows	A simulated car side, sampled with nuScenes' beam spacing. At 8 m it forms a full cluster from raw points or 5 cm voxels, but none with 10 cm voxels. At 15 m and beyond, the gap between laser rings (0.35 m and more) is wider than DBSCAN's 0.25 m radius, so no setting finds it.
Why / what it means	With 10 cm voxels a flat surface keeps only about 13 points per 0.25 m circle, below the required 17, so cars could never be detected (test2: 0%). We now cluster the new sweep at 5 cm while the scene flow keeps 10 cm. It also explains why dense fakes must be denser than real far objects: the paper's setting cannot see a real object at 15 m or more.

4.4 Why some cars and cyclists are missed

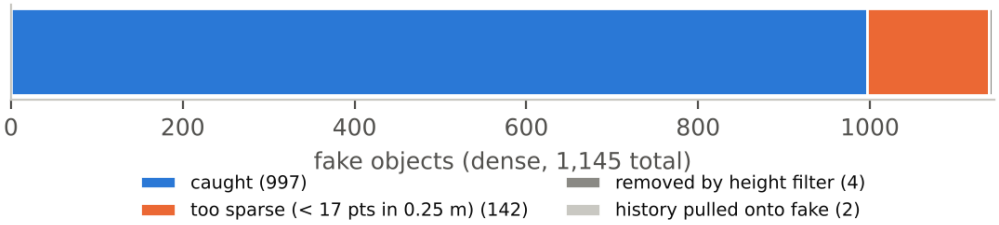


Figure 12. Outcome for every dense fake object in the full run.

What it shows	997 of 1,145 fakes were caught. 142 were 'too sparse': on their own they never reach 17 points within 0.25 m. 4 were removed by the height filter and 2 by the pull effect.
Why / what it means	Almost all misses come from the copied object, not from the memory or the scene flow. Some 200-point chunks of cars (for example a thin front or rear view) and some cyclists are spread too thinly. The paper's attack data was denser, which is the most likely reason for its higher car and cyclist rates.

4.5 False alarms and the two filters

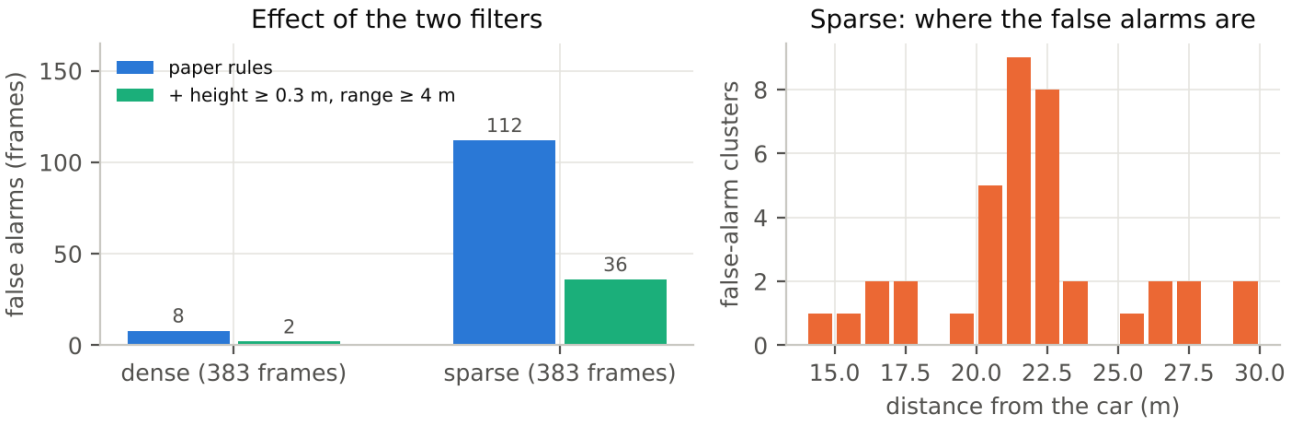


Figure 13. False alarms before and after the filters (left) and where the sparse ones occur (right).

What it shows	Dense: 8 → 2 frames. Sparse: 112 → 36 frames. The remaining sparse false alarms are thin strips (about 0.3 × 1.5 m, 0.5 m tall) concentrated around 20–23 m from the car.
Why / what it means	**Height ≥ 0.3 m** removes flat strips from one laser ring on a curb or road bump: each sweep samples them at a slightly different spot, so they look new. **Distance ≥ 4 m** removes the ring just outside the 3 m zone where we drop the car's own body points; as the car drives, that ring has no history. Both filters are ours, not the paper's, so we report results with and without them. The remaining sparse false alarms are most likely ground that our single-plane ground removal leaves at about 20 m on sloped roads, joined into clusters by the looser 0.75 m radius.

4.6 Runtime

Building the memory took 26.8 s per keyframe on average (it includes the new pair fits) and each check 23.7 s, on a laptop T1200. A deployed detector needs one memory update and one check per sweep, so about 50 s, against the paper's 2.1 s (0.7 s with larger voxels) on an RTX 2080. The gap comes from 10× more optimisation steps, a slower GPU, and not implementing the paper's Scene Flow Approximation (Sec. 5.3). Neither version is real-time (a LiDAR delivers a sweep every 0.05–0.1 s), which the paper also lists as its main limitation. Section 4.7 shows that shorter history or larger voxels cut this by 2–7× without losing detection.

4.7 Ablations (paper Fig. 7, 8, 9)

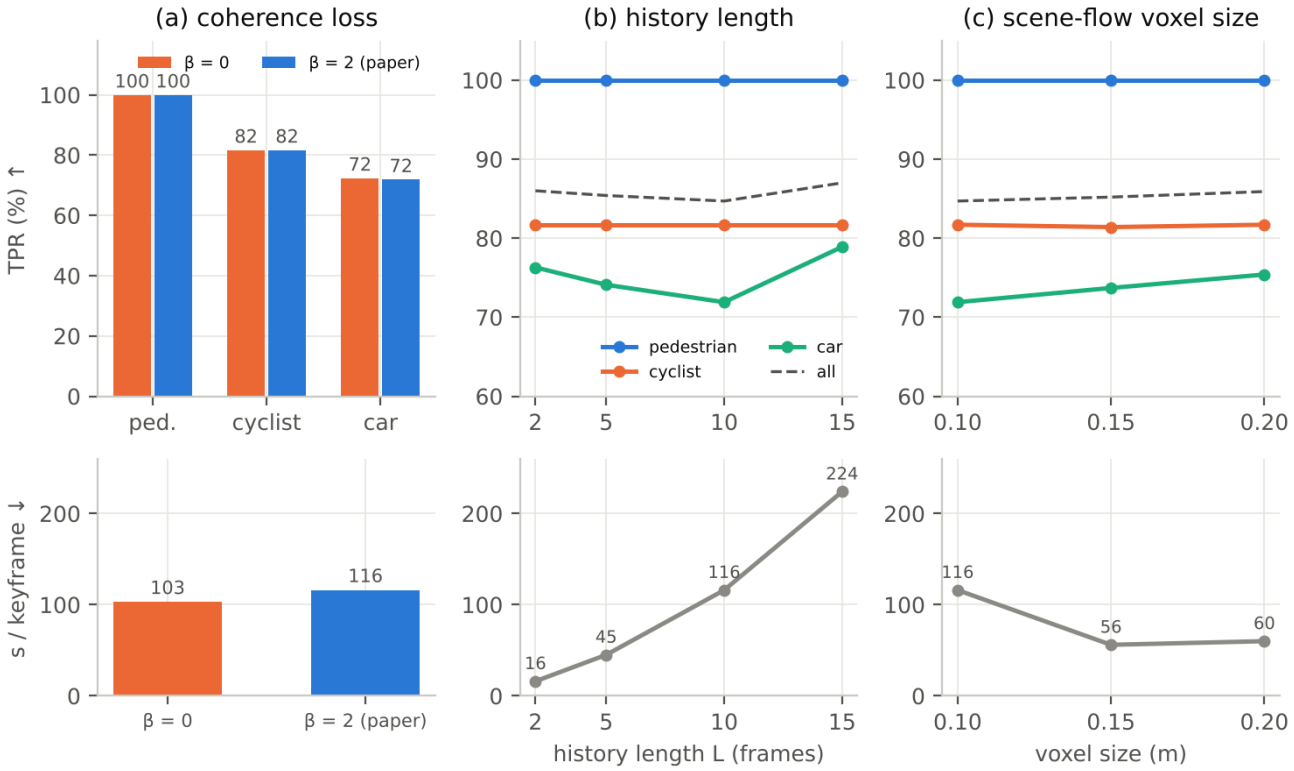


Figure 14. Detection (top) and time per keyframe (bottom) when one setting is changed; 60 keyframes spread over all 10 scenes, same fakes in every run.

What it shows	Each run changes one setting of the main run (β = 2, L = 10, voxel 0.10 m). Pedestrians stay at 100% and cyclists at 81–82% in every run; only cars move (72–79%). False alarms were 0 of 60 frames in every run except the main run (1 of 60). Time per keyframe is one memory build plus three checks.
Why / what it means	**(a) Coherence (paper Fig. 7):** not reproduced as a detection gain: 84.8% without vs 84.7% with. The paper shows the loss stops the scene flow from pulling memory onto a fake; in our pipeline the 0.5 m Chamfer cutoff (section 4.2) already prevents that, so coherence only improves flow accuracy (4.1). **(b) History length (paper Fig. 8):** reproduced: the paper finds 'nearly consistent results from 2 to 15'; ours range 84.7–87.0% while time grows 14× (16 → 224 s). **(c) Voxel size (paper Fig. 9):** half reproduced: time halves (116 → 56 s) as in the paper, but detection does not drop (paper: ~87% of the original accuracy at 0.7 s). Our detection runs on its own 5 cm copy of the new sweep (section 4.3), so the voxel size only affects the scene flow.

How sure are these? 60 frames is enough to see large effects but not small ones. 0 false alarms in 60 frames still allows up to 6% (95% range); the car rate rests on about 57 fakes per run, so differences under about 10 points (for example 72% vs 79%) are within noise. Fake positions can also shift slightly between runs, because fakes are only placed where the memory is empty and the memory changes with L and the voxel size.

5. Differences from the paper and limitations

- **Attack data.** The paper's dense data is private; ours is simulated by copying real objects. Only 10 pedestrians and 6 cyclists in nuScenes-mini meet the dense rules, so those shapes are reused often.
- **Dataset.** The dense comparison uses the same 10 nuScenes v1.0-mini scenes as the paper. The paper's sparse attack used 355 frames from the 150-scene v1.0-trainval validation set; ours uses the 10 mini scenes.
- **No simulator, no physical attack.** Like the paper, all attacks are injected into recorded LiDAR data by software; neither work tests a real laser spoofer or the car's driving response.
- **Baselines.** CARLO and 3D-TC2 were not re-run; their numbers are copied from the paper.

- **Not tested.** The paper's Appendix C failure case (a fake glued to a real object): we always place fakes in free space.
- **Our additions.** 300 steps, lr 3e-3, Chamfer cutoff 0.5 m, 5 cm detection voxels, and the two filters are tuned or added by us. Results are reported with and without the filters.
- **Ablations** (section 4.7) use 60 of the 383 keyframes to keep each run under 3 hours, so they show trends, not precise differences.
- **Scene-flow numbers** (Fig. 8, 9) come from a synthetic street, not from nuScenes, because nuScenes has no ground-truth scene flow.

6. What is left

Status. The method, both full evaluations (dense and sparse), the DBSCAN sweeps and the three ablations are complete. Only the items below remain to polish the report.

To finish the report

- [ ] **Example pictures:** add one 'too sparse' car, one caught pedestrian and one sparse false-alarm strip from *results/full/figures/misses* and *results/full/figures_sparse/misses*.
- [ ] **Title page details:** author, course and supervisor.

Optional extensions

- [ ] Better ground removal (fit the ground in small patches) and rescore the sparse run; this targets most of the remaining sparse false alarms.
- [ ] An 'adjacent' injection mode to reproduce the paper's Appendix C failure case.
- [ ] The paper's Scene Flow Approximation (Sec. 5.3) to reduce runtime.
- [ ] Repeat the dense run with 2–3 different seeds to show run-to-run variation.
- [ ] Check the 0.35 m dense radius on scenes it was not chosen on (for example a few v1.0-trainval scenes).
- [ ] Run the ablations on all 383 keyframes to make the small differences in Fig. 14 reliable.

Appendix: reproducing these numbers

All commands run from the project folder (one line each in PowerShell).

```
python scripts/run_nuscenes.py --dataroot data/nuscenes --out results/full/dense.csv --save-clouds

python scripts/rescore.py results/full/dense.csv
python scripts/rescore.py results/full/dense.csv --min-height 0 --min-range 0 --out results/full/dense_paperonly.csv

python scripts/run_nuscenes.py --dataroot data/nuscenes --attack sparse --out results/full/sparse.csv --save-clouds

python scripts/rescore.py results/full/sparse.csv --eps 0.75 --min-samples 9

python scripts/make_figures.py results/full/dense_paperonly.csv results/full/dense_rescored.csv

python scripts/inspect_misses.py results/full/dense_clouds
python scripts/diag_sfe.py

python scripts/sweep_dbscan.py results/full/dense_clouds --name dense --outdir results/full/figures

python scripts/run_nuscenes.py --dataroot data/nuscenes --sample-every 6 --out results/abl/L2.csv --set L=2
(same for --set L=5, L=15, beta=0, input_voxel=0.15, input_voxel=0.2)

python scripts/ablation.py "L=2=results/abl/L2.csv" "L=5=results/abl/L5.csv" "L=10=results/full/dense_rescored.csv"
"L=15=results/abl/L15.csv" --name history_length --x 2 5 10 15
```